

Applications of Binary Tree

Expression Tree & Huffman Coding

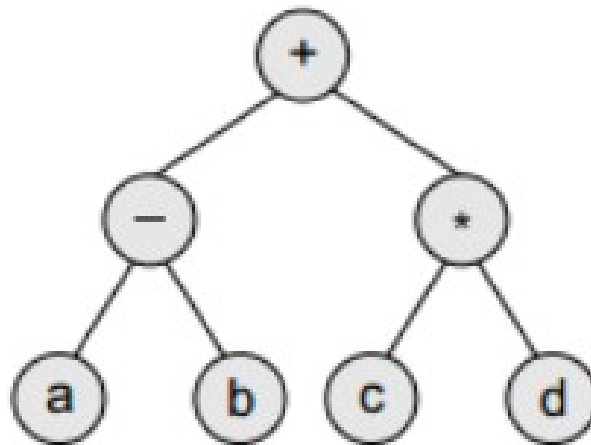
Prof. Siddharth Shah

Expression Tree

- Expression tree is a strictly binary tree in which each internal node corresponds to operator and each leaf node corresponds to operand.
- Binary trees are widely used to store algebraic expressions. For example, consider the algebraic expression given as:

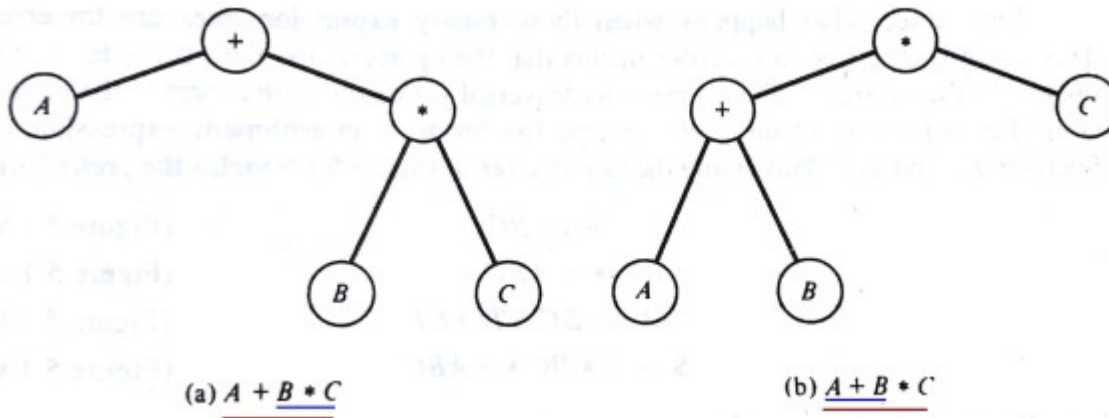
$$\text{Exp} = (a - b) + (c * d)$$

- This expression can be represented using a binary tree as shown in figure below:



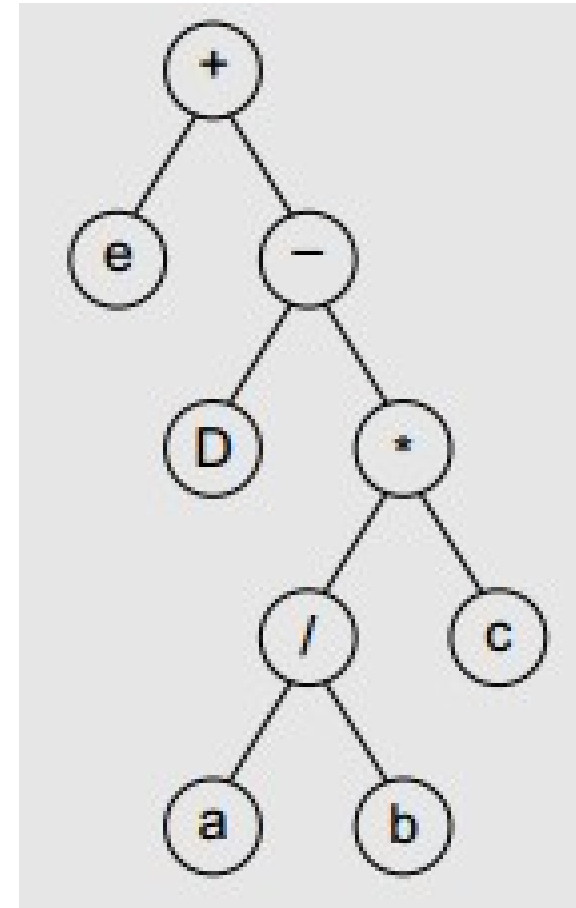
Traversing an Expression Tree

- The Preorder traversal of a binary expression tree yields the prefix form of the expression.
- The Postorder traversal of a binary expression tree yields the postfix form of the expression.
- The Inorder traversal of a binary expression tree yields the infix form of the expression with the ordering of the operations implied by the structure of the tree, as such tree does not contain parenthesis.
- For example, two expression trees given below, when traversed in inorder, generate same expression i.e. **A + B * C**. But the evaluation of each of them is different



Assignment

1. Extract the infix expression it represents
2. Find the corresponding prefix and postfix expressions
3. Evaluate the infix expression, given $a = 30$, $b = 10$, $c = 2$, $d = 30$, $e = 10$



Huffman Coding

- Huffman coding is an encoding algorithm developed by David A. Huffman that is widely used as a lossless data compression technique.
- The idea is to assign variable-length codes to input characters; lengths of the assigned codes are based on the frequencies of corresponding characters.
- The most frequent character gets the smallest code and the least frequent character gets the largest code.
- The variable-length codes assigned to input characters are Prefix Codes, means the codes (bit sequences) are assigned in such a way that the code assigned to one character is not prefix of code assigned to any other character.
- This is how Huffman Coding makes sure that there is no ambiguity when decoding the generated bit stream.

Prefix Codes

- Let there be four characters **a**, **b**, **c** and **d**, and their corresponding variable length codes be **00**, **01**, **0** and **1**.
- This coding leads to ambiguity because code assigned to **c** is prefix of codes assigned to **a** and **b**.
- If the compressed bit stream is **0001**, the decompressed output may be “**cccd**” or “**ccb**” or “**acd**” or “**ab**”.

Generating Huffman Coding

There are mainly two major parts in Huffman Coding

1. Build a Huffman Tree from input characters.
2. Traverse the Huffman Tree and assign codes to characters.

Step 1 - Build a Huffman Tree

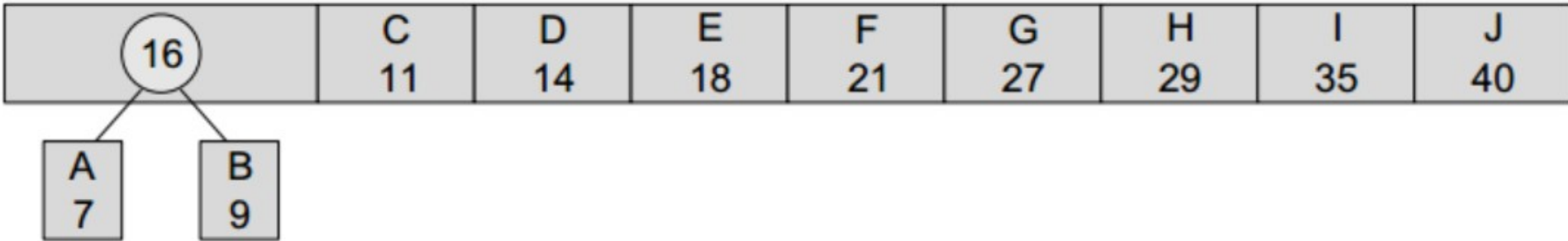
Input is array of unique characters along with their weight or frequency of occurrences and output is Huffman Tree.

1. Create a leaf node for each character. Add the character and its weight or frequency of occurrence to the priority queue.
2. Repeat Steps 3 to 5 while the total number of nodes in the queue is greater than 1.
3. Remove two nodes that have the lowest weight (or highest priority).
4. Create a new internal node by merging these two nodes as children and with weight equal to the sum of the two nodes' weights.
5. Add the newly created node to the queue.

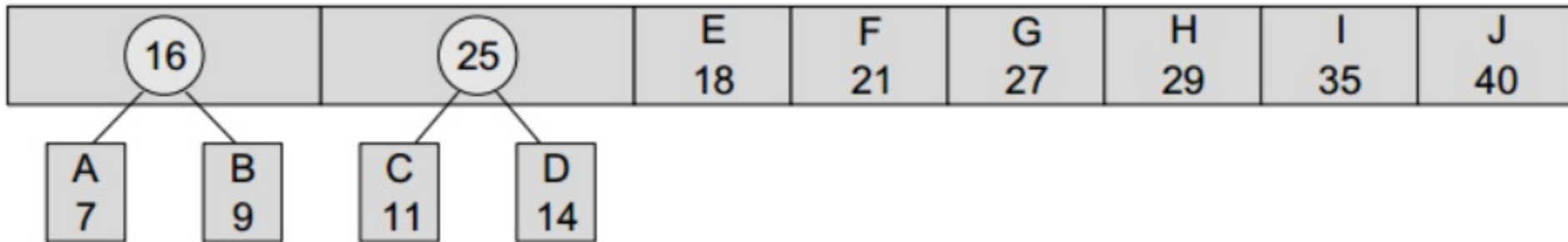
Example

A	B	C	D	E	F	G	H	I	J
7	9	11	14	18	21	27	29	35	40

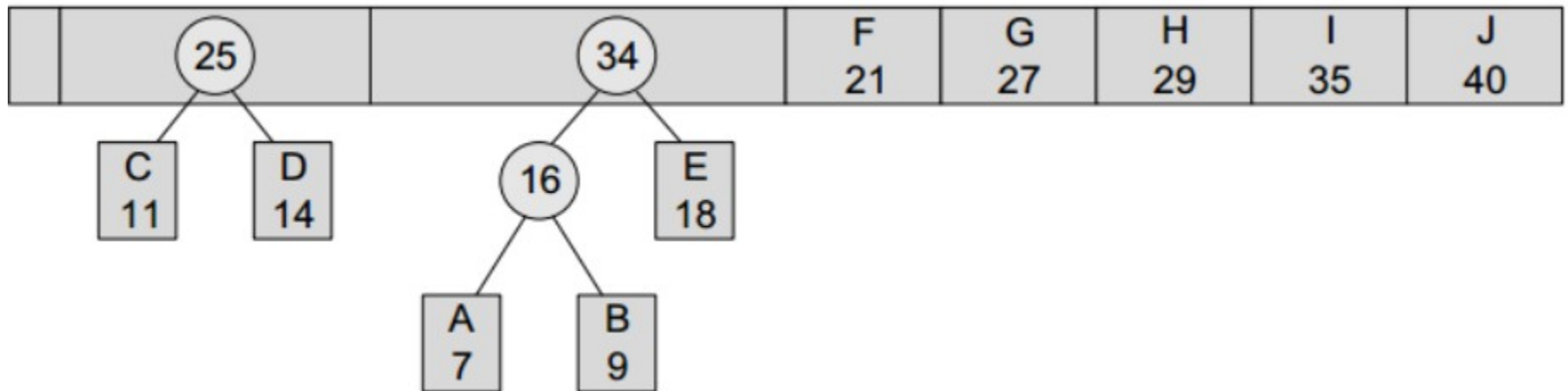
Example



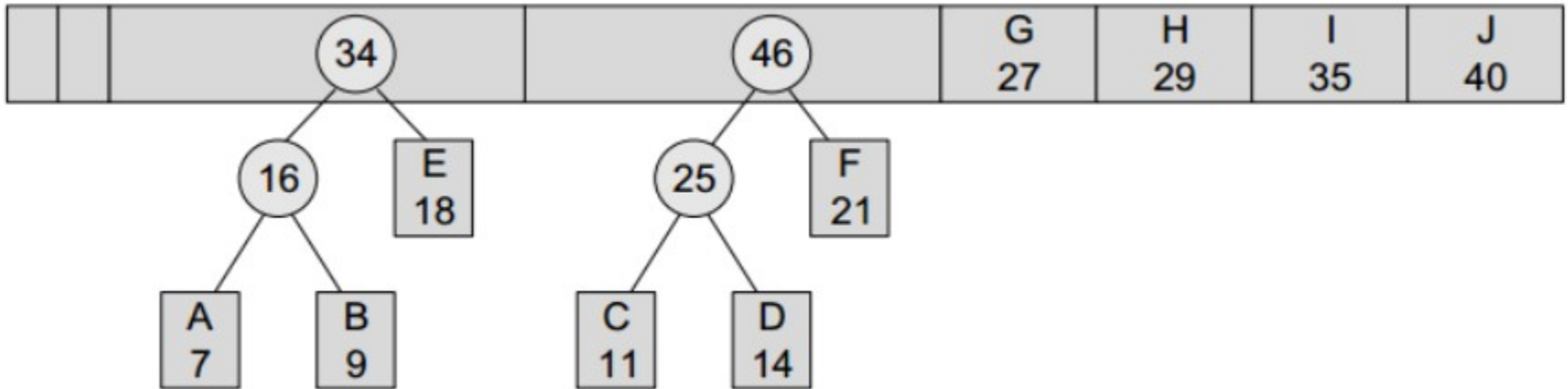
Example



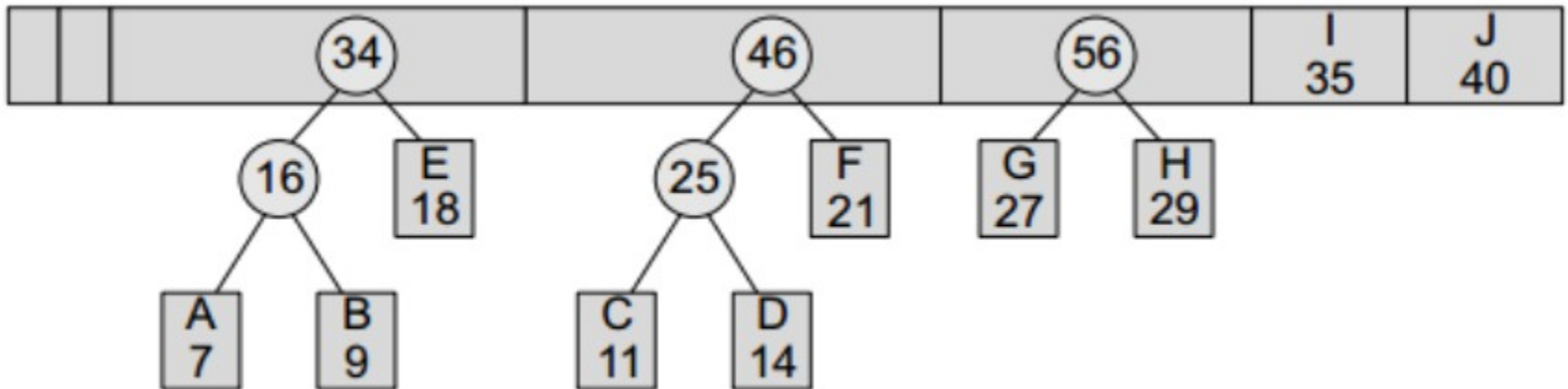
Example



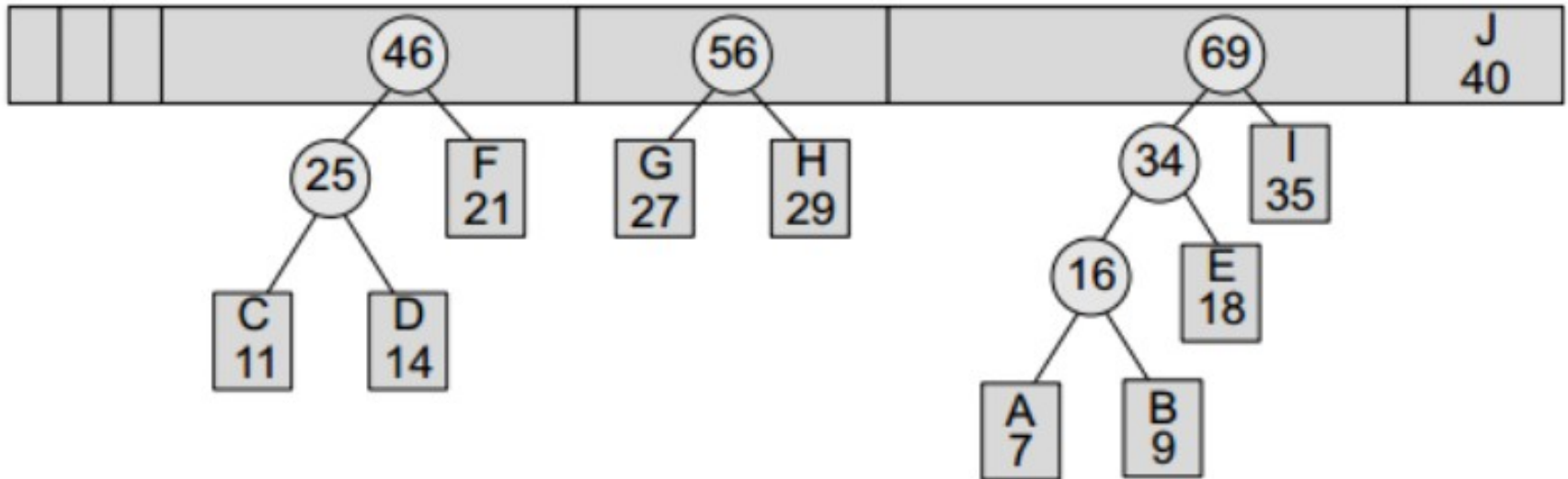
Example



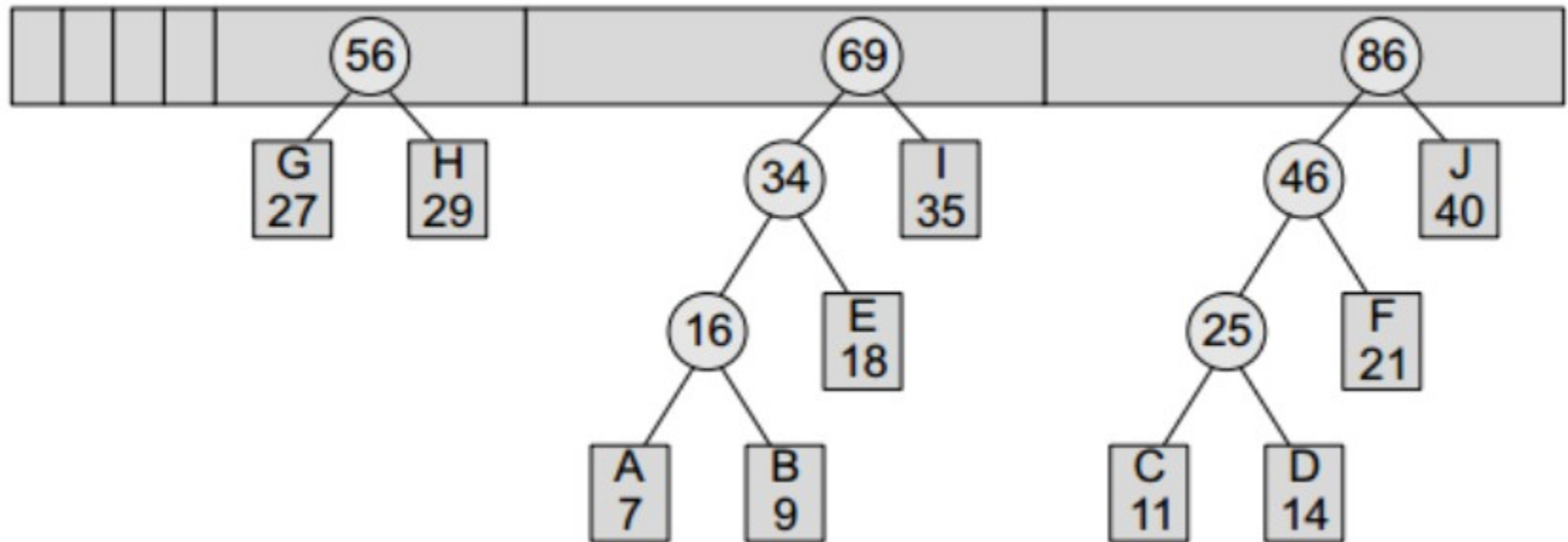
Example



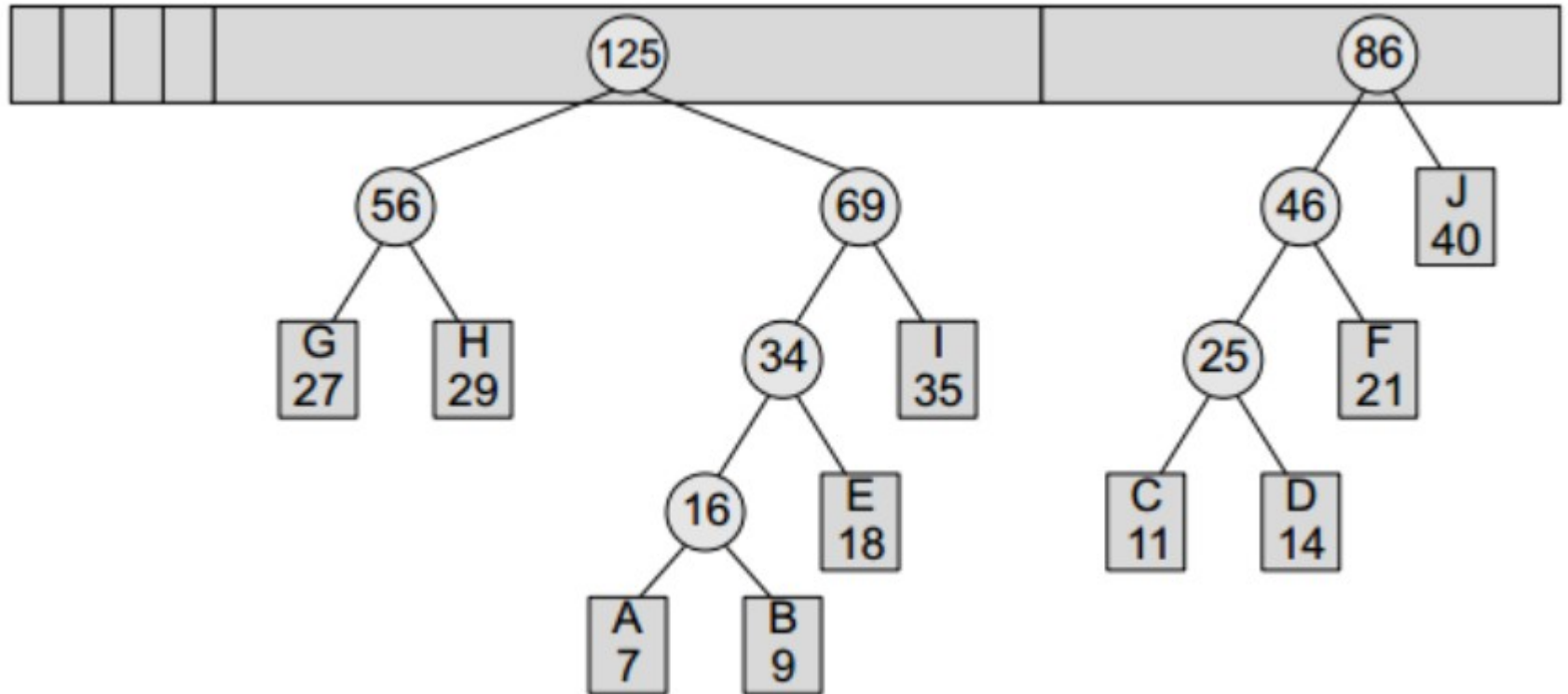
Example



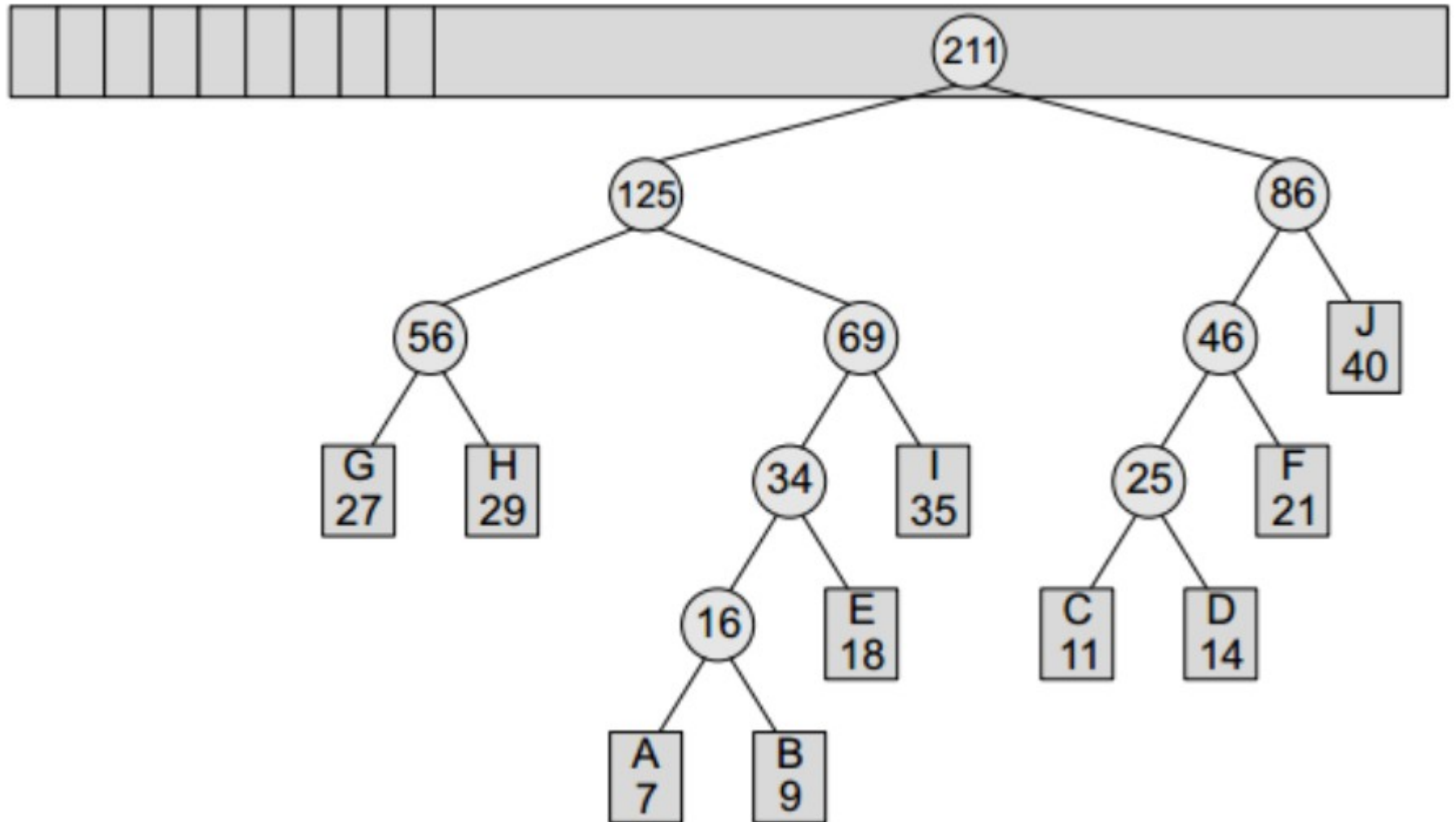
Example



Example



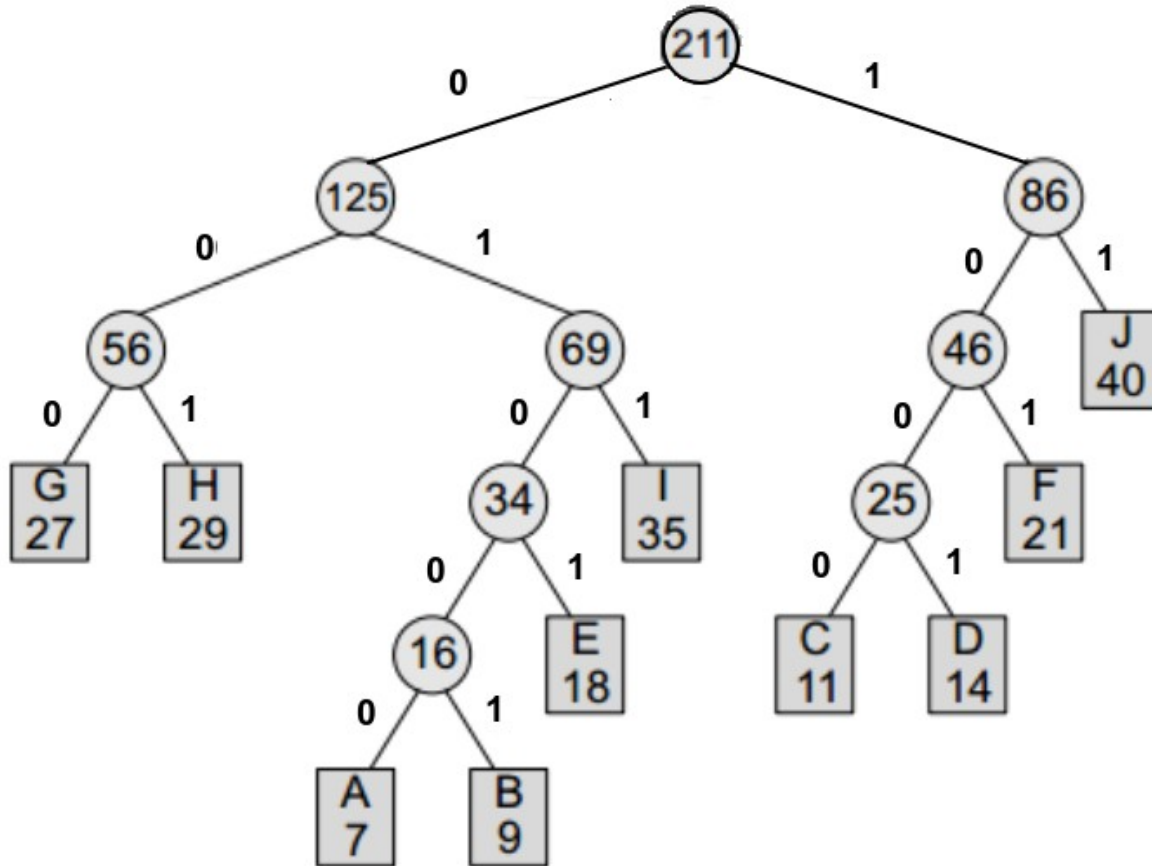
Example



Step 2 – Assign Codes

- In the Huffman tree, circles contain the cumulative weights or frequency of occurrences of their child nodes.
- Every left branch is coded with 0 and every right branch is coded with 1.
- So, the characters A, B, C, D, E, F, G, H, I and J are coded as shown in Table.

Step 2 – Assign Codes



Character	Code
A	01000
B	01001
C	1000
D	1001
E	0101
F	101
G	000
H	001
I	011
J	11

Assignment

Construct a Huffman tree for below given alphabets with their respective frequency. Also assign codes to each of the alphabets using Huffman tree constructed.

Alphabet	A	B	C	D	E	F
Frequency	25	9	17	5	14	10

Properties of a Binary Tree

- The maximum number of nodes at level i of a binary tree is 2^i .
- Maximum number of nodes in a binary tree of height ' h ' is $2^{h+1} - 1$.
- In a Binary Tree with N nodes, minimum possible height is $\text{ceiling}(\text{Log}_2(N+1)-1)$.
- A Binary Tree with L leaves has at least $\text{ceiling}(\text{Log}_2 L) + 1$ levels
- In Binary tree where every node has 0 or 2 children, number of leaf nodes is always one more than nodes with two children.
- Minimum number of nodes in a strictly binary tree is $2h + 1$, where h is the height of a tree.